# A Python Tool for Texas Hold'em

## Probability Estimation & Decision Analysis

Austin Lou & Chang-An Lo

*AMS 561 Final Project*

# What is Texas Hold'em?

*A Popular card game*

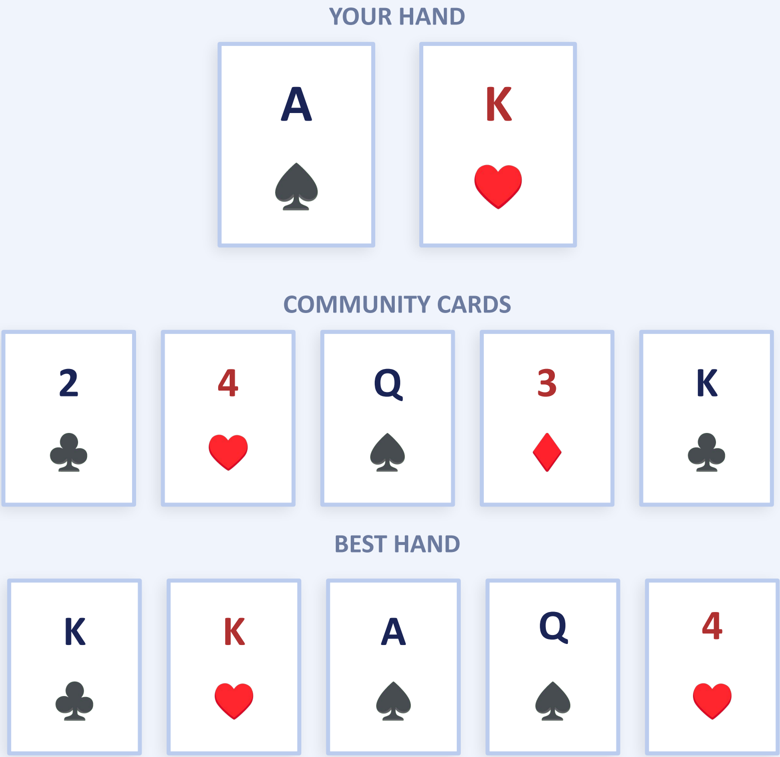
Each player gets **2 hole cards**

**5 community cards** are then dealt face-up on the board in 4 **betting stages**

Players form the best 5-card hand from any combination of their 2 hole cards and the 5 community cards.

### Why did we pick this game?

- Involves probability and incomplete information
- Billions of possible card combinations
- Connects simulation, data analysis & strategy



# Game Stages

*How the board evolves from preflop to river*

### Preflop

**0 community cards**

Each player receives 2 hole cards. First betting stage.

→

### Flop

**3 community cards**

First three community cards revealed. Second betting stage.

→

### Turn

**4 community cards**

Fourth community revealed. Third betting stage.

→

### River

**5 community cards**

Final community card revealed. Exact hand rankings determined, last betting stage.

## Big Questions

- Which of the 169 starting hand categories have the highest preflop win rates (heads-up)?
- How much advantage does suited-ness have against offsuit hands of the same ranks?
- How does a starting hand's equity change as more community comes out?

# Project Scope

*What are we building and why?*

### Hand Evaluator

Given hole cards and all community cards, decide on winning hand.

### Win Probability Calculator

Given hole cards and the known community cards, estimate P(win), P(tie), and P(loss) for

### Decision Support

Run simulations to compile mathematical data on the equity of hole cards against n number of opponents to guide analysis.

### Visualization

Produce tables and plots showing how starting hands, board states, and number of opponents affect equity.

**Python Tools we used: NumPy · Pandas · Matplotlib**

# Our Approach: Hand Evaluator

*Ranking any 5-card combination*

## The Challenge

From 7 available cards (2 hole + 5 community), our hand evaluator has to find the best 5-card hand and correctly rank it against every opponent's best hand.

## 169 Starting Hand Categories

13 Pocket Pairs — AA, KK, QQ, ...

78 Suited — AKs, AQs, KQs, ...

78 Offsuit — AKo, AQo, KQo, ...

## Hand Rankings (High → Low)

| Hand | Description |
|---|---|
| Royal Flush | A K Q J 10 suited |
| Straight Flush | 5 suited consecutive cards |
| Four of a Kind | Four of same rank |
| Full House | Three of same rank + pair |
| Flush | Five cards of same suit |
| Straight | Five consecutive cards |
| Three of a Kind | Three of same rank |
| Two Pair | Two different pairs |
| One Pair | Two of same rank |
| High Card | None of the above |

# Our Approach: Monte Carlo Simulation

*Estimating probabilities through repeated sampling*

## Why Not Exact Enumeration?

| | |
|---|---|
| **Preflop (heads-up):** | C(48,5) ≈ 1.7M combinations |
| **3 opponents:** | C(48,9) ≈ 1.7 billion |
| **5 opponents:** | C(48,13) ≈ 150 billion |

## Monte Carlo Approach

1. Remove known cards from deck (hole + community)
2. Randomly sample remaining positions from the deck
3. Evaluate hands to record win/tie/loss
4. Repeat N = 100,000 times
5. P(win) = wins ÷ N

We double checked Monte Carlo outputs for heads-up against exact enumeration and decided on N=100,000

# Implementation: Hand Evaluator

*Who wins?*

## Design Decisions

### Card Representation

Each card is encoded as (rank, suit). We used 2-14 for ranks for simplicity.  For example Ace of Spades would be (14,s).

### Combination Enumeration

We use itertools.combinations to check all C(7,5) = 21 five-card subsets per hand, returning the highest-ranking one.

### Pattern Detection

Flush: 5 of same suit. Straight: sorted unique ranks span 4. Pairs/trips/quads: rank frequency counts.

### Comparison

Each hand returns a tuple (rank_class, tiebreakers...), for which Python tuple comparison handles the tie-breaking.

## Core Logic (Pseudocode)

```
def evaluate_hand(cards):
  best = None
  for combo in combinations(
      cards, 5):
    score = rank_5card(combo)
    best = max(best, score)
  return best

def rank_5card(hand):
  if is_flush and is_straight:
    return (8, high_card)
  if four_of_a_kind:
    return (7, quad_rank)
  ...
  return (0, high_cards)
```

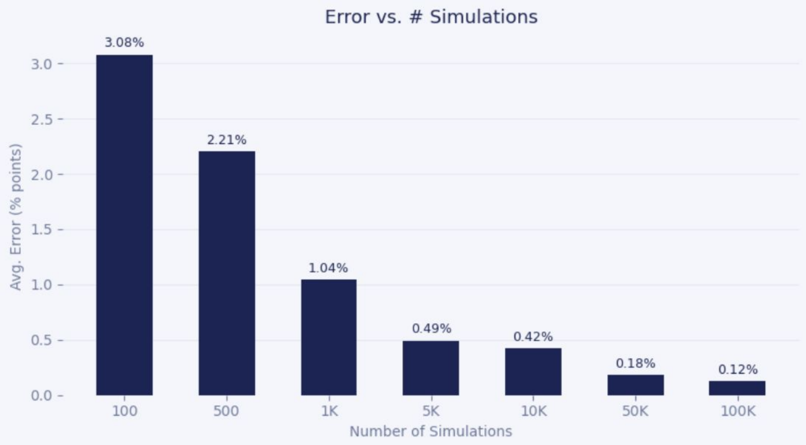
# Implementation: Monte Carlo Simulation

*What's our equity?*

## The Pipeline

| | |
|---|---|
| **Input** | Hole cards, community cards, # opponents |
| **Deck** | Build 52-card deck, take out all known cards |
| **Sample** | np.random.choice to draw cards without replacement |
| **Evaluate** | Call the hand evaluator for each player's best 5 cards |
| **Record** | Win/tie/loss for n simulations |
| **Output** | P(win), P(tie), P(loss) ± confidence interval |

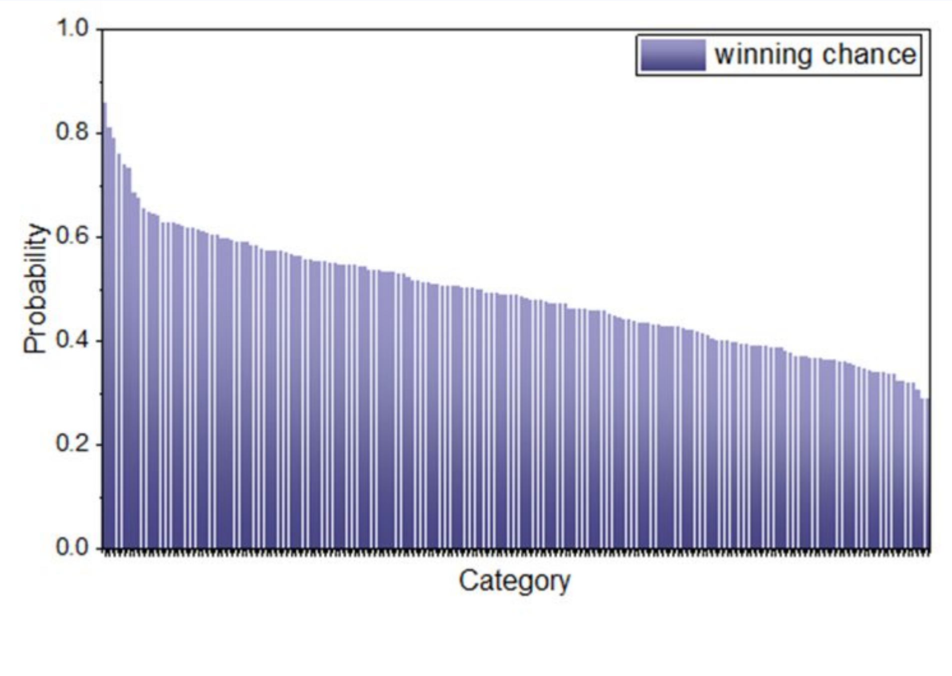
## How accurate are we?




**Parameters we used:**

**n_simulations** = 100,000 | **n_opponents** ∈ {1, 3, 5}

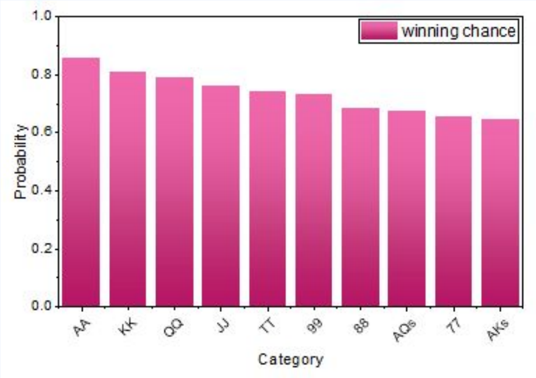
# Q1: Which starting hands are strongest and weakest ?

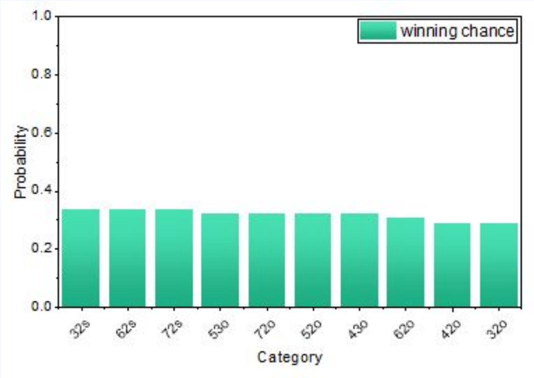



•Premium pairs and high-card hands dominate heads-up equity.
•Weakest hands are mostly low, offsuit, and poorly connected.
•Preflop strength is driven by pair value, card rank, and drawing potential.

# Q1: Which starting hands are strongest and weakest ?

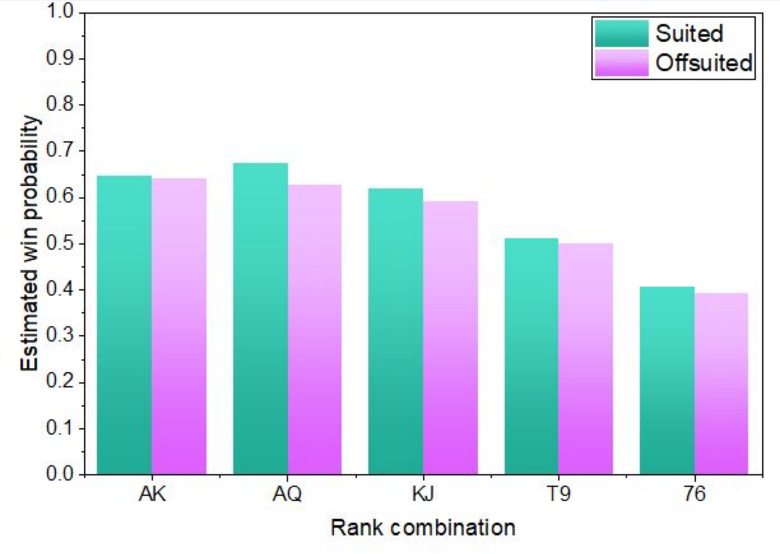
•Low offsuit hands have limited pair, straight, and flush potential.
•Against multiple opponents, their equity collapses quickly.
•This supports folding marginal starting hands in early decisions.



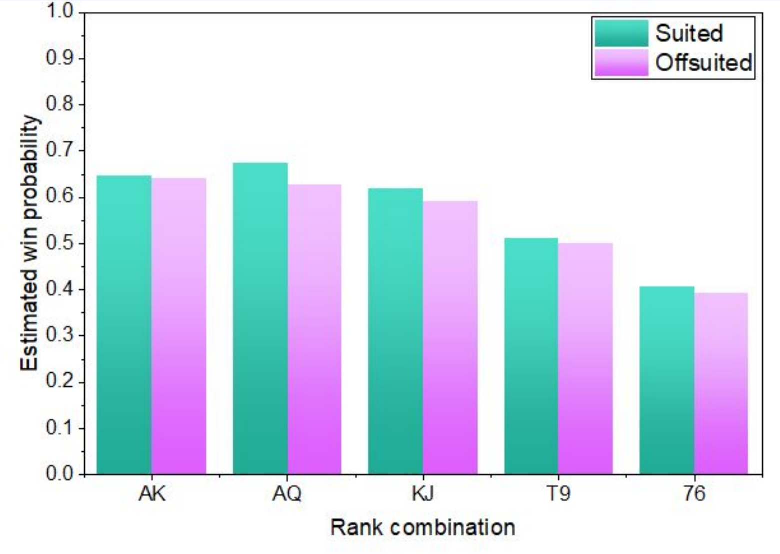

**Takeaway: Premium hands keep the highest equity, but all hands lose win share as the number of opponents increases.**

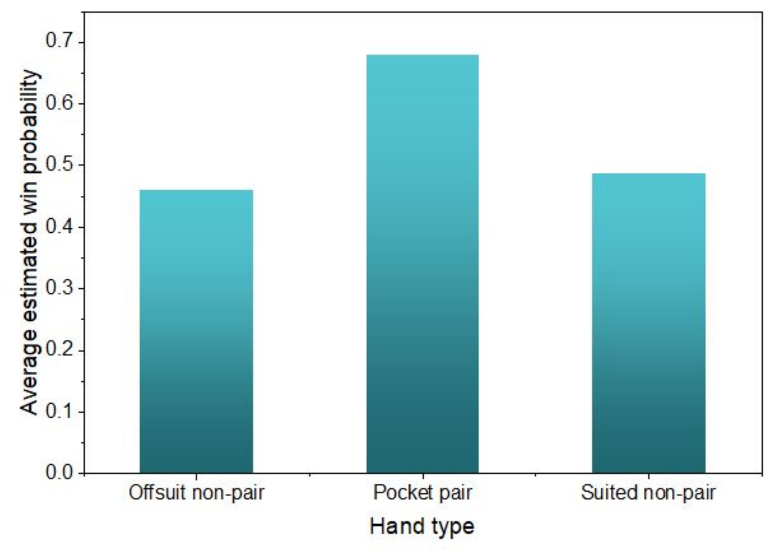
# Q2: How much does suitedness help?




- Suited hands consistently outperform offsuit hands with the same ranks.
- The advantage is modest preflop but meaningful over many hands.
- Suitedness adds flush equity and improves playability after the flop.

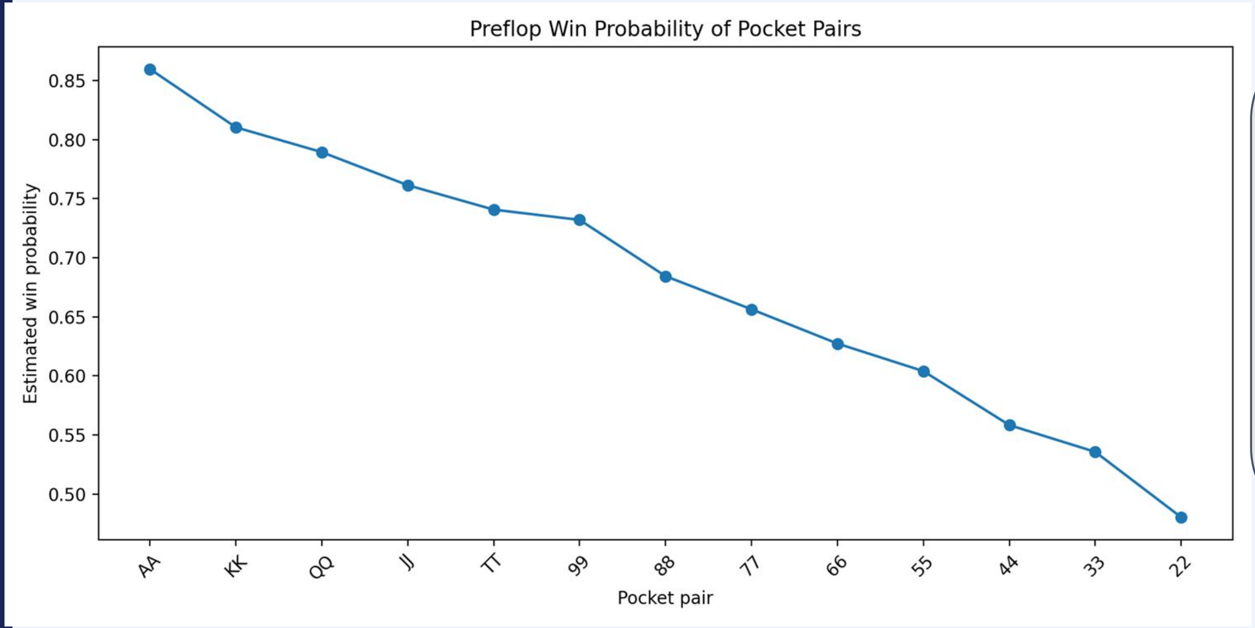
# Q2: How much does suitedness help?



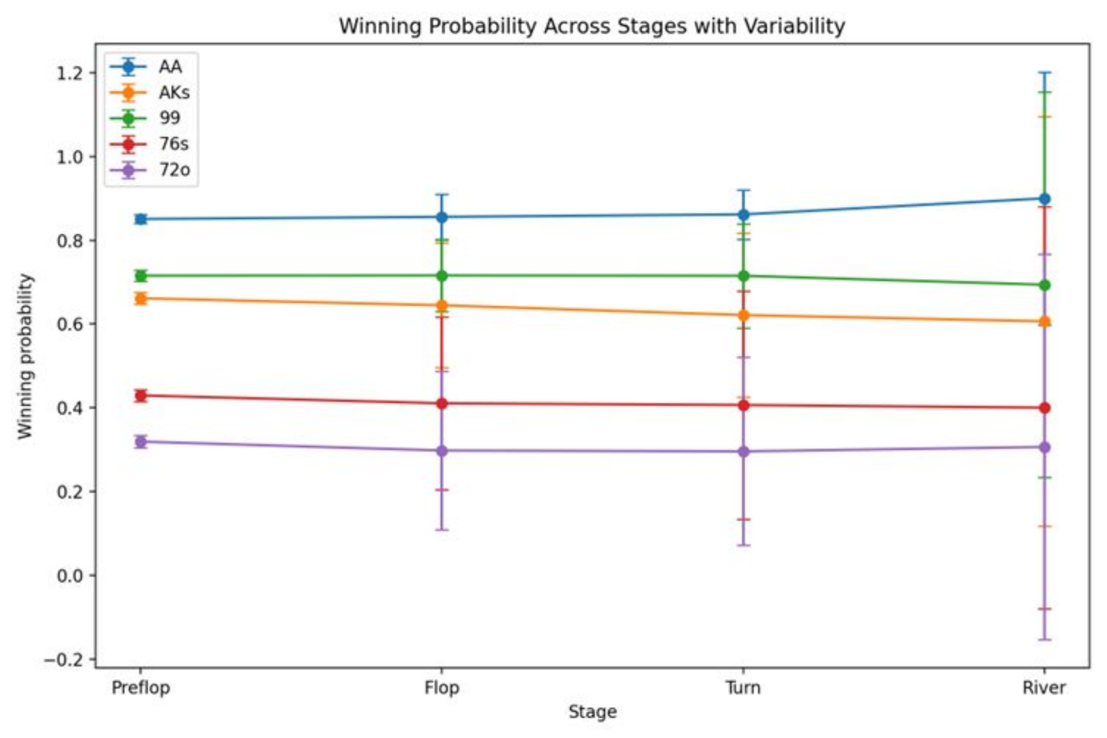

**Takeaway: Suitedness provides a consistent equity boost; the effect is most useful when paired with high rank or connectivity.**

# Q3: How do pocket pairs rank from AA to 22?



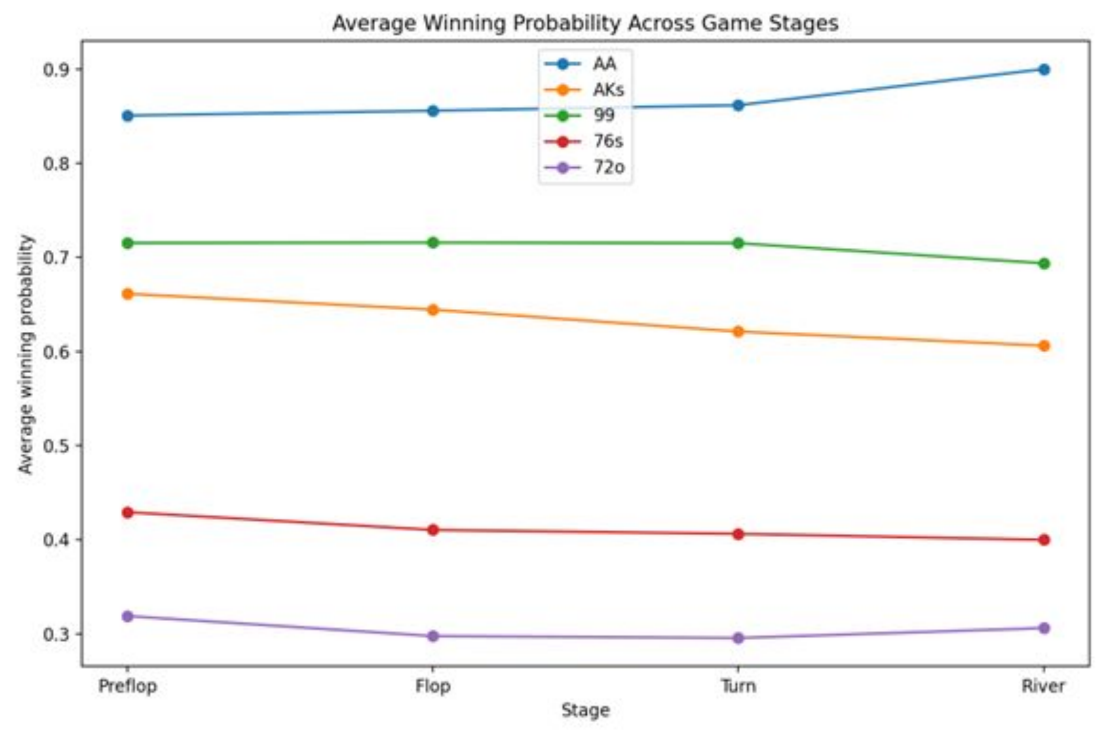

- Pocket-pair equity decreases steadily from AA to 22.
- High pairs often win unimproved; low pairs need to improve more often.
- Pairs are strong preflop, but vulnerability increases in multiway pots.

# Q4: Dynamic winning chance - average win probability across stages



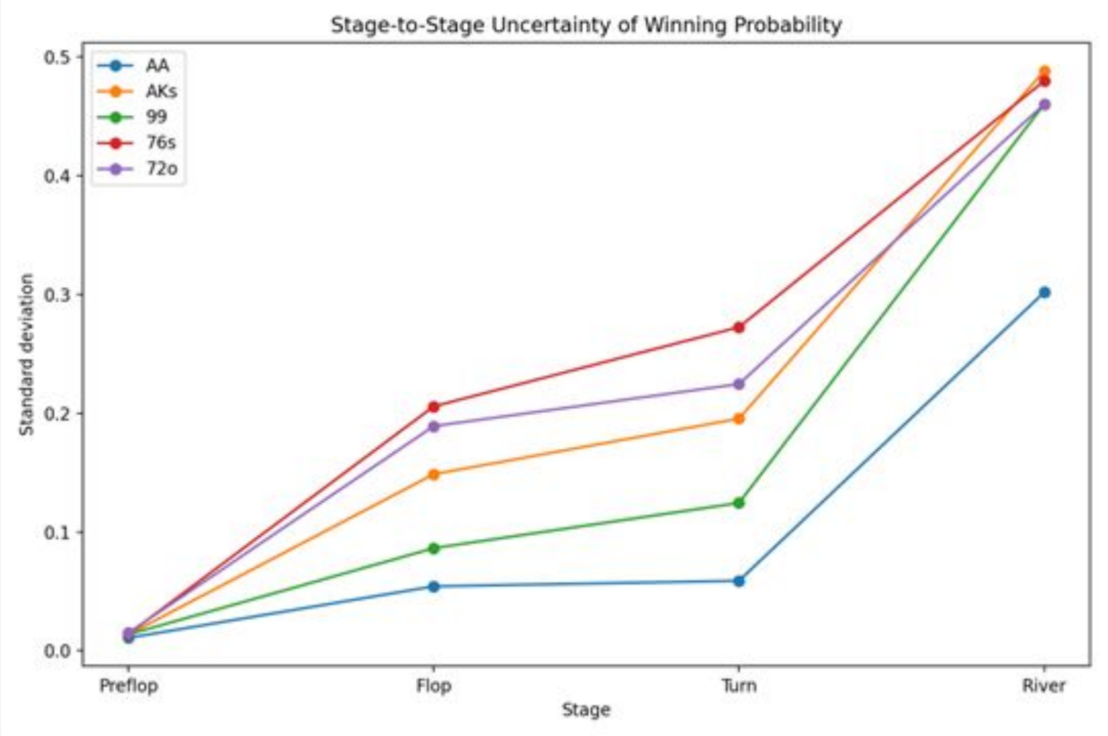

- Win probability becomes more certain as more board cards are revealed.
- Strong starting hands may lose equity if the board improves opponents.
- Stage-by-stage simulation shows how incomplete information narrows over time.

# Q4: Dynamic winning chance - average win probability across stages




- Flop and turn cards can sharply change hand equity.
- River equity becomes deterministic once all cards are known.
- Decision quality improves when simulations update with known community cards.

# Q4: Dynamic winning chance - average win probability across stages




- Preflop decisions rely on average equity across unknown boards.
- Postflop decisions depend strongly on board texture and opponent count.
- A useful tool should update probabilities after every betting stage.

# Thank You

Any Questions For Us?

♠ ♥ ♦ ♣